یکپارچه‌ساختن COMPARE-AND-SWAP

در معماری‌های اینتل x86، دستور زبان اسمبلی cmpxchg برای پیاده‌سازی دستورالعمل compare_and_swap() استفاده می‌شود. به‌منظور اجبار اجرای یکپارچه، پیشوند lock برای قفل گذرگاه استفاده می‌شود در حالی که عملوند مقصد به‌روزرسانی می‌گردد. شکل کلی این دستورالعمل به‌صورت زیر است:

```
lock cmpxchg <destination operand>, <source operand>
```

۶-۵ قفل‌های Mutex

راه‌حل‌های سخت‌افزاری مسأله‌ی بخش بحرانی که در بخش ۶-۴ ارائه شدند، پیچیده بوده، همین‌طور عموماً برای برنامه‌نویسان کاربردی قابل دستیابی نیستند. به جای آن، طراحان سیستم‌عامل، ابزار نرم‌افزاری سطح بالاتری برای حل مسأله‌ی بخش بحرانی می‌سازند. ساده‌ترین این ابزارها **قفل انحصار متقابل**[1] است. (در حقیقت، اصطلاح ***mutex*** اختصار عبارت ***mut***ual ***ex***clusion می‌باشد.) از قفل انحصار متقابل برای حفاظت بخش‌های بحرانی و از این رو، ممانعت از شرایط رقابت استفاده می‌کنیم. یعنی، پردازه بایستی قبل از ورود به بخش بحرانی بایستی قفل کسب کند؛ موقعی که از بخش بحرانی خود خارج می‌شود، آن را رها می‌کند. تابع acquire() قفل را به‌دست می‌آورد و تابع release() قفل را رها می‌سازد، آن‌گونه که در شکل ۶-۱۰ به تصویر کشیده می‌شوند.

```
while (true) {

        acquire lock

        critical section

        release lock

        remainder section

}
```

شکل ۶-۱۰ راه‌حل مسأله‌ی بخش بحرانی با استفاده از قفل‌های انحصار متقابل.

قفل انحصار متقابل دارای یک متغیر منطقی available می‌باشد که مقدار آن نشان می‌دهد، آیا قفل موجود است یا خیر. اگر قفل موجود باشد، فراخوانی acquire() موفق می‌شود و پس از آن، قفل قابل دسترسی نیست. پردازه‌ای که به کسب قفل غیر موجود اقدام می‌کند مسدود می‌شود تا این که قفل رها گردد.

تعریف تابع acquire() به صورت زیر است:

1. Mutex lock

```
acquire() {
        while (!available); /* busy wait */
        available = false;
}
```

تعریف تابع ()release به صورت زیر است:

```
release() {
        available = true;
}
```

فراخوانی‌های توابع ()acquire و یا ()release بایستی به صورت یکپارچه انجام شوند. بنابراین، قفل‌های انحصار متقابل می‌تواند با استفاده از عملیات CAS مطرح شده در بخش ۶-۴ پیاده‌سازی شوند و تشریح این تکنیک به عنوان یک تمرین برای شما کنار گذاشته می‌شود.

رقابت برای قفل[1]

قفل‌ها رقابتی[2] و یا غیررقابتی[3] هستند. یک قفل **رقابتی** محسوب می‌شود در صورتی که یک نخ در حالی که سعی می‌کند قفل را کسب کند مسدود شود. اگر موقعی که یک نخ برای کسب قفل تلاش می‌کند در دسترس باشد، قفل **غیررقابتی** محسوب می‌شود. قفل‌های رقابتی می‌توانند رقابت شدید (تعداد نخ نسبتاً زیادی تلاش می‌کنند که قفل را کسب کنند) و یا رقابت پایین (تعداد نخ نسبتاً می برای کسب قفل تلاش می‌کنند) داشته باشند. تعجب‌آور نیست که قفل‌های رقابتی کارایی کلی برنامه‌های همروند را کاهش می‌دهند.

منظور از «فاصله‌ی کوتاه[4]» چیست؟

قفل‌های چرخشی غالباً به‌عنوان گزینه‌ی انتخابی راهکار قفل‌گذاری در سیستم‌های چندپردازنده می‌باشد زمانی که قفل برای یک فاصله‌ی کوتاهی نگه داشته می‌شود. اما *فاصله‌ی کوتاه* دقیقاً چیست؟ با این فرض که انتظار برای یک قفل مستلزم دو برگردان متن است—یک برگردان متن برای انتقال نخ به حالت انتظار و برگردان متن دوم برای بازیابی نخ منتظر به‌محض این که قفل در دسترس باشد—قاعده‌ی کلی این است که از یک قفل چرخشی استفاده شود، اگر قفل برای یک فاصله‌ی کمتری از دو برگردان متن نگه داشته خواهد شد.

این تکرار حلقه‌ی مستمر به‌وضوح در یک سیستم چندبرنامگی واقعی یک مسئله است که یک هسته‌ی پردازنده‌ی منفردی مابین پردازه‌های زیادی مشترک است. مشغله‌ی انتظار[5] دورهای پردازنده را نیز به‌هدر می‌دهد که یک پردازه‌ی دیگری می‌تواند به‌صورت مفید و کارآمد از آن‌ها استفاده کند. (ما در بخش ۶-۶ راهبردی را بررسی کردیم که با خواب‌سپردن موقت پردازه‌های منتظر و سپس بیدارکردن به‌محض فراهم شدن قفل از مشغله‌ی انتظار اجتناب می‌کرد.)

1. Lock contention
2. Contended
3. Uncontended
4. Short duration
5. Busy-waiting

نوع قفل انحصار متقابلی هم که ما مطرح می‌کنیم یک **قفل چرخشی**[1] نامیده می‌شود، چرا که پردازه در حالی که به انتظار موجود بودن قفل است «می‌چرخد». (همین موضوع را در رابطه با نمونه مثال‌های کدی که دستورالعمل compare_and_swap() را نمایش می‌دادند، دیدیم.) با این وجود، قفل‌های چرخشی یک مزیتی هم دارند، از این نظر که در موقعی که یک پردازه بایستی به‌ناچار برای یک قفل منتظر بماند هیچ برگردان متنی لازم نیست، برگردان‌های متنی که می‌تواند زمان قابل توجهی ببرد. در شرایط خاصی بر روی سیستم‌های چندهسته‌ای، قفل‌های چرخشی حقیقتاً یک گزینه‌ی ترجیحی برای قفل‌گذاری هستند. اگر قرار است که قفل برای فاصله‌ی کوتاهی نگه داشته شود، یک نخ می‌تواند در یک هسته‌ی پردازشی «بچرخد» در حالی که نخ دیگری، بخش بحرانی خودش را در هسته‌ی دیگری به‌انجام می‌رساند. در سیستم‌های رایانش چندهسته‌ای مدرن، قفل‌های چرخشی به‌صورت گسترده در بسیاری از سیستم‌عامل‌ها استفاده می‌شوند.

در فصل ۷ بررسی می‌کنیم که چگونه قفل‌های انحصار متقابل می‌توانند برای حل مسائل کلاسیک همگامی استفاده شوند. همچنین، بحث می‌کنیم که چگونه قفل‌های انحصار متقابل و قفل‌های چرخشی در سیستم‌عامل‌های متعدد، همین‌طور در نخ‌های پوزیکس به‌کار گرفته می‌شوند.

۶-۶ سمافور

همان‌گونه که پیشتر متذکر شدیم، قفل‌های انحصار متقابل عموماً ساده‌ترین ابزار همگامی به حساب می‌آیند. در این بخش، یک ابزار قوی‌تری را بررسی می‌کنیم که به رغم شباهت رفتاری که به قفل انحصار متقابل دارد، می‌تواند روش‌های پیچیده‌ای را برای همگامی فعالیت‌های پردازه‌ها در اختیار آنها بگذارد.

سمافور S، جدا از مقداردهی اولیه، متغیر صحیحی است که تنها از طریق دو عملیات استاندارد یکپارچه قابل دسترسی است: wait() و signal(). سمافورها توسط دانشمند کامپیوتر هلندی، Edsger Dijkstra معرفی شدند عملیات wait() در ابتدا با P (از واژه‌ی هلندی *proberen*، به معنی تست) و عملیات signal() با V (از واژه‌ی *verhogen*، به معنی افزایش) مصطلح بود. عملیات wait() به صورت زیر تعریف می‌شود:

```
wait(S) {
        while S <= 0; // busy wait
        S--;
}
```

تعریف عملیات signal() به صورت زیر است:

```
signal(S) {S++;}
```

تمامی تغییرات عملیات wait() و signal() بر روی مقدار صحیح سمافور بایستی به صورت یکپارچه اجرا شوند. یعنی، لحظه‌ای که پردازه‌ای مقدار سمافور را تغییر می‌دهد، هیچ یک از پردازه‌های دیگر نمی‌توانند همزمان مقدار همان سمافور را تغییر بدهند. به علاوه، در مورد عملیات wait()، بایستی تست مقدار صحیح S (S <= 0) و تغییر احتمالی آن (S--) نیز بی‌وقفه اجرا شوند.

1. Spinlock

در بخش ۶-۶-۲ خواهیم دید که چگونه می‌توان این عملیات را پیاده‌سازی کرد. در ابتدا به چگونگی استفاده از سمافورها می‌پردازیم.

۶-۶-۱ کاربرد سمافور

سیستم‌عامل‌ها غالباً سمافورها را به سمافورهای شمارشی و دودویی تفکیک می‌کنند. مقدار **سمافور شمارشی**[1] می‌تواند در دامنه‌ی نامحدودی تغییر کند. مقدار **سمافور دودویی**[2] فقط بین ۰ و ۱ تغییر می‌کند. بنابراین، سمافورهای دودویی مشابه قفل‌های انحصار متقابل رفتار می‌کنند. در حقیقت، در سیستم‌هایی که قفل‌های انحصار متقابل فراهم نمی‌شوند، در عوض سمافورهای دودویی می‌توانند برای فراهم‌ساختن انحصار متقابل استفاده شوند.

از سمافورهای شمارشی می‌توان برای کنترل دسترسی یک منبع با تعداد نمونه‌های متناهی استفاده کرد. سمافور به تعداد منابع در دسترس مقداردهی می‌شود. هر پردازه‌ای که می‌خواهد از منبعی استفاده نماید یک عملیات wait() بر روی سمافور انجام می‌دهد (کاهش یک واحدی شمارش). زمانی هم که پردازه‌ای منبعی را رها می‌کند، یک عملیات signal() انجام می‌دهد (افزایش یک واحدی شمارش). لحظه‌ای که شمارش سمافور به ۰ می‌رسد، تمامی منابع در حال استفاده هستند. بعد از آن، پردازه‌هایی که می‌خواهند از منبعی استفاده نمایند مسدود خواهند شد تا این که شمارش را بزرگ‌تر از ۰ بیابند.

می‌توانیم برای حل مسائل مختلف همگامی نیز از سمافورها استفاده کنیم. برای مثال، دو پردازه را با اجرای همروند در نظر بگیرید: P_1 با دستور S_1 و P_2 با دستور S_2. فرض کنید می‌خواهیم S_2 فقط بعد از تکمیل S_1 اجرا شود. برای پیاده‌سازی این طرح به سادگی اجازه می‌دهیم تا P_1 و P_2 از سمافور مشترک synch با مقدار اولیه‌ی ۰ استفاده کنند. در پردازه‌ی P_1، دستورات زیر را وارد می‌کنیم:

```
S1;
signal(synch);
```

در پردازه‌ی P_2، دستورات زیر را وارد می‌کنیم:

```
wait(synch);
S2;
```

چون synch مقدار اولیه‌ی ۰ را دارد، P_2 فقط در صورتی دستور S_2 را اجرا خواهد کرد که P_1 دستور signal(synch) را اجرا کرده باشد که آن هم بعد از اجرای دستور S_1 مقدور است.

۶-۶-۲ پیاده‌سازی سمافور

به خاطر دارید که پیاده‌سازی قفل‌های انحصار متقابل بحث شده در بخش ۶-۵ از مشغله‌ی انتظار رنج می‌برد. تعاریف عملیات wait() و signal() سمافور که مطرح شدند، همان مسأله را

1. Counting semaphores
2. Binary semaphore

دارند. برای غلبه بر این مسئله می‌توانیم تعریف عملیات wait() و signal() را به این صورت اصلاح کنیم: لحظه‌ای که پردازه‌ای عملیات wait() را اجرا می‌کند و مقدار سمافور را غیرمثبت می‌یابد، بایستی منتظر بماند. با این وجود، پردازه به جای این که مشغله‌ی انتظار پیشه کند، خودش را *معلق* می‌سازد. عملیات تعلیق، یک پردازه را در یک صف انتظار مرتبط با سمافور گذاشته و حالت پردازه به حالت انتظار تبدیل می‌شود. سپس کنترل به زمانبند پردازنده منتقل می‌شود که پردازه‌ی دیگری را برای اجرا انتخاب می‌کند.

پردازه‌ای که به انتظار سمافور S معلق می‌شود، بایستی با اجرای عملیات signal() پردازه‌های دیگر، دومرتبه کار خود را شروع نماید. پردازه با عملیات wakeup() که حالت پردازه را از انتظار به آماده‌باش تغییر می‌دهد، دوباره از سر گرفته می‌شود. سپس پردازه در صف آماده گذاشته می‌شود. (بسته به الگوریتم زمانبندی پردازنده امکان دارد که پردازنده از پردازه‌ی در حال اجرا پس‌گرفته شده و در اختیار پردازه‌ای که هم‌اکنون به صف آماده رسیده قرار بگیرد و یا پردازه در همان‌جا منتظر نگه داشته شود.)

جهت پیاده‌سازی سمافورها مطابق تعریف اخیر، سمافور را به صورت زیر تعریف می‌کنیم:

```
typedef struct {
        int value;
        struct process *list;
} semaphore;
```

هر سمافور دارای یک مقدار صحیح value و یک list از پردازه‌ها می‌باشد. زمانی که پردازه‌ای به ناچار منتظر سمافوری می‌ماند، به لیست پردازه‌ها افزوده می‌شود. عملیات signal()، یکی از پردازه‌های لیست پردازه‌های منتظر را خارج نموده و آن را بیدار می‌کند.

حال می‌توان عملیات wait() سمافور را به صورت زیر تعریف کرد:

```
wait(semaphore *S) {
                S->value--;
                if (S->value < 0) {
                        add this process to S->list;
                        sleep();
                }
}
```

و عملیات signal() سمافور می‌تواند به صورت زیر تعریف شود:

```
signal(semaphore *S) {
                S->value++;
                if (S->value <= 0) {
                        remove a process P from S->list;
                        wakeup(P);
                }
}
```

عملیات ()sleep، پردازه‌ای را که آن را احضار کرده، معلق می‌سازد. عملیات (P)wakeup، اجرای پردازه‌ی معلق P را از سر می‌گیرد. این دو عملیات به عنوان فراخوانی‌های سیستمی پایه‌ای توسط سیستم‌عامل فراهم می‌شوند.

توجه کنید به رغم این که مقدار سمافور در تعریف معمول سمافورها با مشغله‌ی انتظار هرگز منفی نمی‌شد، ولی با این پیاده‌سازی، مقادیر منفی سمافورها نیز امکان‌پذیر است. اگر مقدار سمافوری منفی باشد، قدر مطلق آن، تعداد پردازه‌های منتظر سمافور را مشخص می‌کند. این واقعیت، پیامد پیاده‌سازی عملیات ()wait است که در آن، هر پردازه سمافور را یک واحد کاهش داده و سپس آن را بررسی می‌کند.

لیست پردازه‌های منتظر را می‌توان به راحتی با قراردادن یک فیلد لینک در هر بلاک کنترل پردازه (PCB) پیاده‌سازی کرد. هر سمافور یک مقدار صحیح و یک اشاره‌گر به یک لیست از PCBها خواهد داشت. یک روش اضافه و حذف پردازه‌های لیست که انتظار محدود را نیز تضمین می‌نماید استفاده از یک صف FIFO می‌باشد که، اشاره‌گرهای سر و ته صف نیز در سمافور قرار می‌گیرند. با این وجود، در حالت کلی می‌توان از هر راهبردی برای لیست استفاده کرد. کاربرد درست سمافورها به یک راهبرد صف‌بندی خاص لیست‌های سمافور بستگی ندارد.

همان‌گونه که عنوان شد مهم است که عملیات سمافور، اجرای یکپارچه داشته باشند. بایستی تضمین کنیم که هیچ دو پردازه‌ای نمی‌توانند همزمان عملیات ()wait و ()signal را بر روی سمافور یکسان اجرا کنند. این یک مسأله‌ی بخش بحرانی است؛ و در یک محیط تک‌پردازنده می‌توانیم آن را به سادگی با منع وقفه‌ها در طول زمان اجرای عملیات ()wait و ()signal حل کنیم. این طرح در محیط تک‌پردازنده کار می‌کند، چون همین که وقفه‌ها منع می‌شوند، دستورالعمل‌های پردازه‌های متفاوت نمی‌توانند اجرای یک در میان داشته باشند. تنها پردازه‌ای که فعلاً در حال اجراست، اجرا می‌شود تا این که دومرتبه وقفه‌ها فعال شده و زمانبند باز هم کنترل را به دست آورد.

در یک محیط چندهسته‌ای بایستی وقفه‌های یکایک هسته‌های پردازشی غیرفعال شوند. در غیر این صورت، دستورالعمل‌های پردازه‌های متفاوت (که در هسته‌های متفاوتی اجرا دارند) ممکن است به ترتیب دلخواهی یک در میان اجرا شوند. از کار انداختن وقفه‌های هر یک از هسته‌ها کار دشواری است و می‌تواند کارایی سیستم را به‌طور جدی پایین بیاورد. بنابراین، سیستم‌های SMP بایستی تکنیک‌های جایگزینی—همچون ()compare_and_swap یا قفل‌های چرخشی—را برای تضمین انجام یکپارچه‌ی عملیات ()wait و ()signal فراهم کنند.

توجه به این نکته اهمیت دارد که مشغله‌ی انتظار را با این تعریف ()wait و ()signal کاملاً حذف نکرده‌ایم. بلکه، مشغله‌ی انتظار را از بخش ورود به بخش‌های بحرانی برنامه‌های کاربردی انتقال داده‌ایم. به علاوه، مشغله‌ی انتظار را به بخش‌های بحرانی عملیات ()wait و ()signal محدود کرده‌ایم که این بخش‌ها کوتاه هستند (اگر درست برنامه‌نویسی شوند، نباید بیش از ده دستورالعمل بشوند). بنابراین، بخش بحرانی تقریباً هرگز اشغال نمی‌گردد و مشغله‌ی انتظار هم به ندرت پیش می‌آید و اگر هم اتفاق بیفتد تنها برای مدت زمان کوتاهی خواهد بود. یک وضعیت کاملاً متفاوت در برنامه‌های کاربردی وجود دارد که ممکن است بخش‌های بحرانی آنها طولانی